

INTER-PROCESS COMMUNICATION USING PIPE

Muthukutti R

241801175

Write a C program to demonstrate Inter-Process Communication (IPC) using a pipe. The program should create a parent and child process using `fork()`. The parent process should send a message to the child process through the pipe, and the child process should read and display the received message.

Input Format:

The input consists of a single integer `n` entered by the user.

Output Format:

- Displays the value sent by the parent process
- Displays the processed value (double of the number) received and computed by the child process

Sample Input:

5

Sample Output:

Parent sent: 5

Child received: 5

Child processed value (double): 10

CODE:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
int main() {  
    int fd[2];  
    pid_t pid;  
  
    char write_msg[] = "Hello from Parent Process";  
    char read_msg[100];  
  
    // Create pipe  
    if (pipe(fd) == -1) {  
        printf("Pipe failed\n");  
        return 1;  
    }  
  
    // Create child process  
    pid = fork();  
  
    if (pid < 0) {  
        printf("Fork failed\n");  
        return 1;  
    }  
}
```

```
// Parent Process
```

```
if (pid > 0) {
```

```
    close(fd[0]); // Close read end
```

```
    write(fd[1], write_msg, strlen(write_msg) + 1);
```

```
    printf("Parent sent: %s\n", write_msg);
```

```
    close(fd[1]); // Close write end
```

```
}
```

```
// Child Process
```

```
else {
```

```
    close(fd[1]); // Close write end
```

```
    read(fd[0], read_msg, sizeof(read_msg));
```

```
    printf("Child received: %s\n", read_msg);
```

```
    close(fd[0]); // Close read end
```

```
}
```

```
return 0;
```

```
}
```

Output:

Enter a number: 5

Parent sent: 5

Child received: 5

Child processed value (double): 10